Programming Hero

PRODUCT ENGINEERING PLAYBOOK

PRODUCT ইঞ্জিনিয়ারিং

সঠিক সমস্যা বাছাই, দায়িত্বশীল ডেলিভারি এবং মাপযোগ্য ফল।

অভিজ্ঞ সফটওয়্যার ইঞ্জিনিয়ারিং সিদ্ধান্তের একটি ব্যবহারিক গাইড।

সমস্যা → সিদ্ধান্ত → ডেলিভারি → শেখা

একটি টেকসই প্রোডাক্ট কীভাবে তৈরি হয়?

- সমস্যার প্রমাণ সংগ্রহ
- ঝুঁকি ও খরচ যাচাই
- ছোট রিলিজ
- বাস্তব ব্যবহার
- পুনর্নির্ধারণ

মূল নীতি

কোড আউটপুট নয়; ব্যবহারকারীর মাপযোগ্য ফলাফলই সফলতার মাপকাঠি।

অধ্যায় ০১ | ইঞ্জিনিয়ারিং বিচারবোধ

ফিচার নয়, ফলাফলকে কেন্দ্র করো

একজন অভিজ্ঞ ইঞ্জিনিয়ার প্রথমেই implementation নিয়ে আলোচনা করেন না। তিনি বোঝেন সমস্যাটি কার, ব্যবসার মূল্য কত এবং কোন সিদ্ধান্তটি এখন নেওয়া জরুরি।

০১

সিদ্ধান্ত নেওয়ার আগে চারটি বাস্তব প্রশ্ন

01

সমস্যার প্রমাণ

ব্যবহারকারী feedback, usage data, নাকি কেবল অভ্যন্তরীণ ধারণা?

02

ব্যবসায়িক প্রভাব

সমাধান হলে retention, completion বা support load-এ কী বদলাবে?

03

ইঞ্জিনিয়ারিং খরচ

নতুন dependency, migration ও maintenance-এর দায় কতটা?

04

ব্যর্থতার পরিণতি

পরিবর্তন ভুল হলে ব্যবহারকারী, data ও operation কীভাবে প্রভাবিত হবে?

মাঠের একটি পরিচিত পরিস্থিতি

কেস: কোর্স প্ল্যাটফর্মে completion কম

কোর্স প্ল্যাটফর্মে completion কম। টিম নতুন dashboard চাইছে। কিন্তু funnel দেখে বোঝা গেল mobile network drop-এর পর progress save হচ্ছে না। নতুন dashboard নয়, আগে state persistence এবং retry handling ঠিক করা দরকার।

অভিজ্ঞতার নিয়ম stakeholder requirement ধরে নিও না। প্রথমে সমস্যাকে যাচাইযোগ্য hypothesis-এ রূপ দাও।

অধ্যায় ০২ | ডিসকভারি ও ডায়াগনসিস

লক্ষণ নয়, মূল কারণ খুঁজে বের করো

ব্যবহারকারীর অভিযোগ একটি signal, root cause নয়। একাধিক evidence source দেখানো ছাড়া solution-এ commit করা অপ্রয়োজনীয় rework তৈরি করে।

ডিসকভারির চারটি evidence layer

01

ব্যবহারকারীর আচরণ

Session replay, interview এবং task observation-এ friction কোথায়?

02

টেলিমেট্রি

কোন device, cohort বা version-এ failure বেশি হচ্ছে?

03

সিস্টেমের অবস্থা

Logs, traces, API latency ও retry pattern কী বলছে?

04

পরিস্থিতিগত সীমা

নেটওয়ার্কভাষা, accessibility এবং device capability কী প্রভাব ফেলছে?

ভালো problem statement কেমন হয়?

দুর্বল বনাম শক্তিশালী

"শিক্ষার্থীরা অসন্তুষ্টএকটি দুর্বল বক্তব্য। এর বদলে: "দুর্বল নেটওয়ার্কে mobile learner-এর progress update হারাচ্ছে; ফলে পরের session-এ আগের পাঠ পুনরায় শুরু করতে হচ্ছে।" এখানে user, context, failure এবং impact স্পষ্ট।

সমাধানের আগে falsifiable hypothesis

Hypothesis উদাহরণ

"Progress update-এ idempotent retry ও local queue যোগ করলে interrupted session-এর পর incorrect resume-এর হার কমবে।" এরপর event definition, baseline এবং rollback trigger নির্ধারণ করো।

মনে রেখো Interview দিয়ে কারণের সূত্র পাওয়া যায়; prevalence প্রমাণ করতে event data ও segment analysis প্রয়োজন।

অধ্যায় ০৩ | অগ্রাধিকার ও স্কোপ

সীমিত সময়ে সঠিক বাজি ধরো

টিম সব backlog item-এই গুরুত্ব রাখে। কিন্তু engineering capacity, operational risk এবং opportunity cost বিবেচনা না করলে roadmap দ্রুত অকার্যকর হয়ে ওঠে।

অগ্রাধিকারের চারটি মানদণ্ড

01

Impact

সমাধান হলে কোন outcome বদলাবে এবং কত ব্যবহারকারী উপকৃত হবে?

02

Confidence

প্রভাবের পক্ষে কতটা নির্ভরযোগ্য evidence আছে?

03

Effort

Build, testing, migration, observability ও future support-এর মোট খরচ কত?

04

Risk

Security, reliability, reversibility এবং dependency risk কী?

একই টিমের তিনটি বিকল্প

কাজ	প্রভাব	বাস্তবায়ন	সিদ্ধান্ত
Progress retry fix	উচ্চ	কম-মাঝারি	আগে করো
নতুন visual theme	অনিশ্চিত	মাঝারি	পরে যাচাই
AI recommendation	সম্ভাবনাময়	উচ্চ	আলাদা discovery

MVP মানে নিরাপত্তা সংকুচিত নয়

ছোট scope, কিন্তু শক্ত ভিত্তি

MVP-তে scope কমবে, কিন্তু critical correctness, privacy ও error handling বাদ যাবে না। সবচেয়ে গুরুত্বপূর্ণ অনুমানটি যাচাই করার মতো ছোট vertical slice বানাও; পুরো system redesign নয়।

মাপকাঠি প্রথম release-এর সাফল্য feature count নয়; গুরুত্বপূর্ণ অনিশ্চয়তা কতটা কমল সেটিই দেখো।

অধ্যায় ০৪ | ইমপ্লিমেন্টেশন ও ডেলিভারি

কোড থেকে production-ready পরিবর্তন

Delivery হলো design handoff বা pull request merge-এর চেয়ে বড় দায়িত্ব। গ্রহণযোগ্য আচরণ, failure path এবং operational ownership শুরুতেই পরিষ্কার হতে হবে।

08

একটি actionable engineering specification

01

Expected behavior

পাঠ শেষ হলে progress update হবে এবং পরের load-এ একই state ফিরবে।

02

Edge case

নেটওয়ার্ক বিচ্ছিন্ন হলে duplicate completion তৈরি হবে না।

03

Acceptance criteria

Reconnect-এর পর data consistency এবং mobile resume flow যাচাই হবে।

04

Release ownership

Feature flag, monitoring dashboard ও on-call escalation path নির্ধারিত থাকবে।

Implementation checklist: শুধু happy path নয়

- [ ] Database change backward-compatible কি না পরীক্ষা করো।
- [ ] Automated test-এ concurrency ও retry case রাখো।
- [ ] Logs-এ sensitive user information ঢুকছে কি না দেখো।
- [ ] Latency এবং failure rate-এর alert threshold সেট করো।
- [ ] Rollback করলে data contract অটুট কি না যাচাই করো।

মূল কথা Production readiness-এর মানে deploy করতে পারা নয়; সমস্যা হলে দ্রুত ক্ষতি সীমিত করতে পারা।

অধ্যায় ০৫ | রিলিজ ও পরিমাপ

ডেটা ছাড়া improvement দাবি কোরো না

কোনো feature চালু হয়েছে বলেই সেটি কার্যকর হয়েছে, এমন নয়। adoption, successful task completion এবং system health একসঙ্গে দেখতে হয়।

চার ধরনের signal একসঙ্গে পর্যবেক্ষণ করো

01

Adoption

Eligible user-এর কতজন feature ব্যবহার করছেন?

02

Outcome

কতজন interruption-এর পর সফলভাবে পাঠ নিয়েছেন?

03

Reliability

Error rate, p95 latency এবং data mismatch বেড়েছে কি?

04

Guardrail

Support ticket, crash ও user complaint-এর হার কী?

উদাহরণ: সংখ্যাকে context-সহ পড়ো

Eligible	Feature used	Task completed	ব্যাখ্যা
১,০০০	৪২০	২১০	৫০%

এখানে ২১০ ÷ ৪২০ = ৫০% হলো feature user-এর completion rate। কিন্তু improvement প্রমাণ করতে previous baseline, cohort mix এবং experiment duration জানতে হবে। সংখ্যাগুলো কাল্পনিক।

Release strategy

ধাপে ধাপে এগোও

আগে internal validation, পরে সীমিত rollout; guardrail স্থিতিশীল থাকলে traffic বাড়াও। adverse trend দেখা দিলে rollout থামানো বা revert করার সিদ্ধান্ত পূর্বনির্ধারিত রেখো।

সতর্কতা Metric নিজে সব বলে না। সঠিক denominator, instrumentation এবং comparison group ছাড়া সেটি বিভ্রান্তিকর হতে পারে।

অধ্যায় ০৬ | পর্যবেক্ষণ ও পুনর্নির্ধারণ

শেখার চক্র থামানো যাবে না

ভালো engineering team শুধু delivery velocity দেখে না। ভুল অনুমান কত দ্রুত শনাক্ত হলো, incident থেকে কী শিখল এবং পরের সিদ্ধান্তে কী বদলাল, সেটিও দেখে।

০৬

Release-এর পরে চারটি decision path

01

Signal শক্তিশালী

Outcome ও guardrail ভালো থাকলে ধাপে ধাপে rollout বাড়াও।

02

Signal অস্পষ্ট

Tracking, sample size এবং user segment পুনরায় পরীক্ষা করো।

03

Outcome খারাপ

Root cause বিশ্লেষণ করে scope বদলাও বা feature revert করো।

04

Risk বৃদ্ধি

Reliability বা privacy impact থাকলে expansion বন্ধ করে incident সামলাও।

সাপ্তাহিক engineering review-এ জিজ্ঞেস করো

- [ ] কোন অনুমানটি বাস্তব ডেটায় ভুল প্রমাণিত হলো?
- [ ] কোন incident বা edge case আমাদের অপ্রস্তুত করেছে?
- [ ] কোন technical debt এখন delivery risk বাড়াচ্ছে?
- [ ] পরের সপ্তাহে একটি সবচেয়ে মূল্যবান পরীক্ষা কী?

মনে রেখো Blameless review মানে accountability বাদ দেওয়া নয়; ব্যক্তিকে নয়, system এবং সিদ্ধান্তপ্রক্রিয়াকে উন্নত করা।

শেষ অধ্যায় | কাজের অনুশীলন

১৪ দিনে একটি ছোট engineering experiment

নিজের পরিচিত product থেকে একটি সীমিত কিন্তু বাস্তব সমস্যা বেছে নাও। বড় feature না বানিয়ে measurable outcome এবং operational safety-সহ একটি পরিবর্তন delivery করো।

০৭

১৪ দিনের execution plan

দিন ১–২	সমস্যা, প্রভাবিত user ও baseline চিহ্নিত	Problem brief
দিন ৩–৪	Interview, logs ও event audit	Evidence summary
দিন ৫–৬	Scope, risk ও acceptance criteria	Decision record
দিন ৭–১০	Small vertical slice build ও test	Working increment
দিন ১১–১২	Controlled rollout ও health check	Metrics snapshot
দিন ১৩–১৪	Outcome review ও পরের সিদ্ধান্ত	Learning memo

নিজের experiment-এর সিদ্ধান্তপত্র

সমস্যা: _____

Baseline ও target: _____

সবচেয়ে বড় ঝুঁকি: _____

Rollback condition: _____

Outcome review date: _____

শেষ কথা পরিশ্রমের সম্মান, ছোট scope, নিরাপদ release এবং evidence-ভিত্তিক সিদ্ধান্তই পরিণত product engineering-এর ভিত্তি।